

Distributed Mutual Exclusion

Introduction

- Mutual exclusion: Concurrent access of processes to a shared resource or data is executed in mutually exclusive manner.
- Only one process is allowed to execute the critical section (CS) at any given time.
- In a distributed system, shared variables (semaphores) or a local kernel cannot be used to implement mutual exclusion.
- Message passing is the sole means for implementing distributed mutual exclusion.

Introduction

- Distributed mutual exclusion algorithms must deal with unpredictable message delays and incomplete knowledge of the system state.
- Three basic approaches for distributed mutual exclusion:
 - 1 Token based approach
 - 2 Non-token based approach
 - 3 Quorum based approach
- Token-based approach:
 - ▶ A unique token is shared among the sites.
 - ▶ A site is allowed to enter its CS if it possesses the token.
 - ▶ Mutual exclusion is ensured because the token is unique.

Introduction

- Non-token based approach:
 - ▶ Two or more successive rounds of messages are exchanged among the sites to determine which site will enter the CS next.
- Quorum based approach:
 - ▶ Each site requests permission to execute the CS from a subset of sites (called a quorum).
 - ▶ Any two quorums contain a common site.
 - ▶ This common site is responsible to make sure that only one request executes the CS at any time.

Preliminaries

System Model

- The system consists of N sites, $S_1, S_2, \dots, S_N$.
- We assume that a single process is running on each site. The process at site S_i is denoted by p_i .
- A site can be in one of the following three states: requesting the CS, executing the CS, or neither requesting nor executing the CS (i.e., idle).
- In the 'requesting the CS' state, the site is blocked and can not make further requests for the CS. In the 'idle' state, the site is executing outside the CS.
- In token-based algorithms, a site can also be in a state where a site holding the token is executing outside the CS (called the *idle token* state).
- At any instant, a site may have several pending requests for CS. A site queues up these requests and serves them one at a time.

Requirements

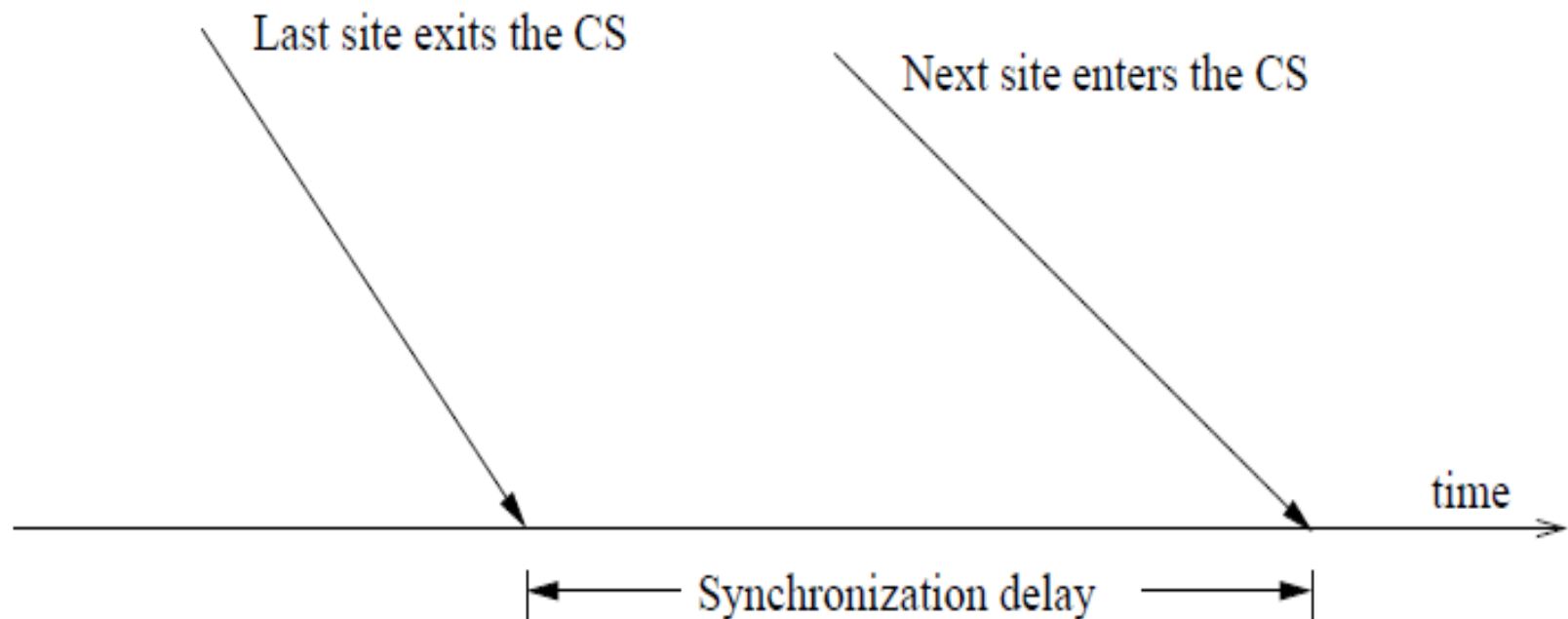
Requirements of Mutual Exclusion Algorithms

- 1 **Safety Property:** At any instant, only one process can execute the critical section.
- 2 **Liveness Property:** This property states the absence of deadlock and starvation. Two or more sites should not endlessly wait for messages which will never arrive.
- 3 **Fairness:** Each process gets a fair chance to execute the CS. Fairness property generally means the CS execution requests are executed in the order of their arrival (time is determined by a logical clock) in the system.

Performance Metrics

The performance is generally measured by the following four metrics:

- **Message complexity:** The number of messages required per CS execution by a site.
- **Synchronization delay:** After a site leaves the CS, it is the time required and before the next site enters the CS (see Figure 1).



Performance Metrics

- **Response time:** The time interval a request waits for its CS execution to be over after its request messages have been sent out (see Figure 2).

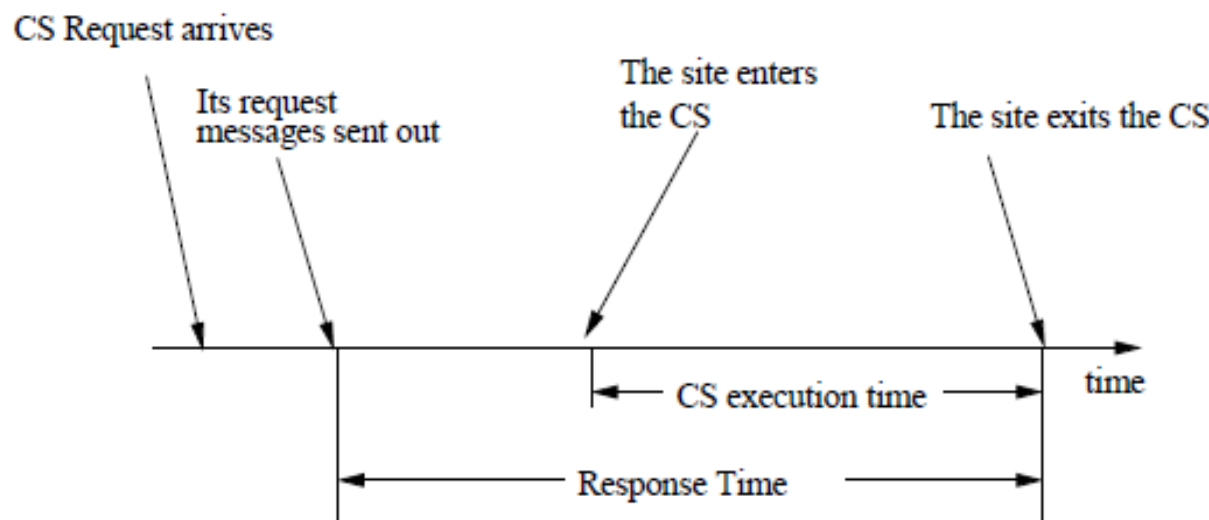


Figure 2: Response Time.

- **System throughput:** The rate at which the system executes requests for the CS.

$$\text{system throughput} = 1 / (SD + E)$$

where SD is the synchronization delay and E is the average critical section execution time.

Performance Metrics

Low and High Load Performance:

- We often study the performance of mutual exclusion algorithms under two special loading conditions, viz., “low load” and “high load”.
- The load is determined by the arrival rate of CS execution requests.
- Under *low load* conditions, there is seldom more than one request for the critical section present in the system simultaneously.
- Under *heavy load* conditions, there is always a pending request for critical section at a site.

Lamport's Algorithm

- Requests for CS are executed in the increasing order of timestamps and time is determined by logical clocks.
- Every site S_i keeps a queue, *request_queue_i*, which contains mutual exclusion requests ordered by their timestamps.
- This algorithm requires communication channels to deliver messages the FIFO order.

The Algorithm

Requesting the critical section:

- When a site S_i wants to enter the CS, it broadcasts a REQUEST(ts_i, i) message to all other sites and places the request on $request_queue_i$. ((ts_i, i) denotes the timestamp of the request.)
- When a site S_j receives the REQUEST(ts_i, i) message from site S_i , places site S_i 's request on $request_queue_j$ and it returns a timestamped REPLY message to S_i .

Executing the critical section: Site S_i enters the CS when the following two conditions hold:

- L1: S_i has received a message with timestamp larger than (ts_i, i) from all other sites.
- L2: S_i 's request is at the top of $request_queue_i$.

The Algorithm

Releasing the critical section:

- Site S_i , upon exiting the CS, removes its request from the top of its request queue and broadcasts a timestamped RELEASE message to all other sites.
- When a site S_j receives a RELEASE message from site S_i , it removes S_i 's request from its request queue.

When a site removes a request from its request queue, its own request may come at the top of the queue, enabling it to enter the CS.

correctness

Theorem: Lamport's algorithm achieves mutual exclusion.

Proof:

- Proof is by contradiction. Suppose two sites S_i and S_j are executing the CS concurrently. For this to happen conditions L1 and L2 must hold at both the sites *concurrently*.
- This implies that at some instant in time, say t , both S_i and S_j have their own requests at the top of their *request_queues* and condition L1 holds at them. Without loss of generality, assume that S_i 's request has smaller timestamp than the request of S_j .
- From condition L1 and FIFO property of the communication channels, it is clear that at instant t the request of S_i must be present in *request_queue_j* when S_j was executing its CS. This implies that S_j 's own request is at the top of its own *request_queue* when a smaller timestamp request, S_i 's request, is present in the *request_queue_j* – a contradiction!

correctness

Theorem: Lamport's algorithm is fair.

Proof:

- The proof is by contradiction. Suppose a site S_i 's request has a smaller timestamp than the request of another site S_j and S_j is able to execute the CS before S_i .
- For S_j to execute the CS, it has to satisfy the conditions L1 and L2. This implies that at some instant in time say t , S_j has its own request at the top of its queue and it has also received a message with timestamp larger than the timestamp of its request from all other sites.
- But *request_queue* at a site is ordered by timestamp, and according to our assumption S_i has lower timestamp. So S_i 's request must be placed ahead of the S_j 's request in the *request_queue_j*. This is a contradiction!

Performance

- For each CS execution, Lamport's algorithm requires $(N - 1)$ REQUEST messages, $(N - 1)$ REPLY messages, and $(N - 1)$ RELEASE messages.
- Thus, Lamport's algorithm requires $3(N - 1)$ messages per CS invocation.
- Synchronization delay in the algorithm is T .

An optimization

- In Lamport's algorithm, REPLY messages can be omitted in certain situations. For example, if site S_j receives a REQUEST message from site S_i after it has sent its own REQUEST message with timestamp higher than the timestamp of site S_i 's request, then site S_j need not send a REPLY message to site S_i .
- This is because when site S_i receives site S_j 's request with timestamp higher than its own, it can conclude that site S_j does not have any smaller timestamp request which is still pending.
- With this optimization, Lamport's algorithm requires between $3(N - 1)$ and $2(N - 1)$ messages per CS execution.

Ricart-Agrawala Algorithm

- The Ricart-Agrawala algorithm assumes the communication channels are FIFO. The algorithm uses two types of messages: REQUEST and REPLY.
- A process sends a REQUEST message to all other processes to request their permission to enter the critical section. A process sends a REPLY message to a process to give its permission to that process.
- Processes use Lamport-style logical clocks to assign a timestamp to critical section requests and timestamps are used to decide the priority of requests.
- Each process p_i maintains the Request-Deferred array, RD_i , the size of which is the same as the number of processes in the system.
- Initially, $\forall i \forall j: RD_i[j]=0$. Whenever p_i defer the request sent by p_j , it sets $RD_i[j]=1$ and after it has sent a REPLY message to p_j , it sets $RD_i[j]=0$.

Description of the Algorithm

Requesting the critical section:

- (a) When a site S_i wants to enter the CS, it broadcasts a timestamped REQUEST message to all other sites.
- (b) When site S_j receives a REQUEST message from site S_i , it sends a REPLY message to site S_i if site S_j is neither requesting nor executing the CS, or if the site S_j is requesting and S_i 's request's timestamp is smaller than site S_j 's own request's timestamp. Otherwise, the reply is deferred and S_j sets $RD_j[i]=1$

Executing the critical section:

- (c) Site S_i enters the CS after it has received a REPLY message from every site it sent a REQUEST message to.

Algorithm

Releasing the critical section:

- (d) When site S_i exits the CS, it sends all the deferred REPLY messages: $\forall j$ if $RD_i[j]=1$, then send a REPLY message to S_j and set $RD_i[j]=0$.

Notes:

- When a site receives a message, it updates its clock using the timestamp in the message.
- When a site takes up a request for the CS for processing, it updates its local clock and assigns a timestamp to the request.

Correctness

Theorem: Ricart-Agrawala algorithm achieves mutual exclusion.

Proof:

- Proof is by contradiction. Suppose two sites S_i and S_j are executing the CS concurrently and S_i 's request has higher priority than the request of S_j . Clearly, S_i received S_j 's request after it has made its own request.
- Thus, S_j can concurrently execute the CS with S_i only if S_i returns a REPLY to S_j (in response to S_j 's request) before S_i exits the CS.
- However, this is impossible because S_j 's request has lower priority. Therefore, Ricart-Agrawala algorithm achieves mutual exclusion.

Performance

- For each CS execution, Ricart-Agrawala algorithm requires $(N - 1)$ REQUEST messages and $(N - 1)$ REPLY messages.
- Thus, it requires $2(N - 1)$ messages per CS execution.
- Synchronization delay in the algorithm is T .

Quorum-Based Mutual Exclusion Algorithms

Quorum-based mutual exclusion algorithms are different in the following two ways:

- 1 A site does not request permission from all other sites, but only from a subset of the sites. The request set of sites are chosen such that $\forall i \forall j : 1 \leq i, j \leq N :: R_i \cap R_j \neq \Phi$. Consequently, every pair of sites has a site which mediates conflicts between that pair.
- 2 A site can send out only one REPLY message at any time. A site can send a REPLY message only after it has received a RELEASE message for the previous REPLY message.

continuation..

Since these algorithms are based on the notion of 'Coterie' and 'Quorums', we next describe the idea of coterie and quorums.

A coterie C is defined as a set of sets, where each set $g \in C$ is called a quorum. The following properties hold for quorums in a coterie:

- **Intersection property:** For every quorum $g, h \in C$, $g \cap h \neq \emptyset$.
For example, sets $\{1,2,3\}$, $\{2,5,7\}$ and $\{5,7,9\}$ cannot be quorums in a coterie because the first and third sets do not have a common element.
- **Minimality property:** There should be no quorums g, h in coterie C such that $g \supseteq h$. For example, sets $\{1,2,3\}$ and $\{1,3\}$ cannot be quorums in a coterie because the first set is a superset of the second.

continuation..

Coterie and quorums can be used to develop algorithms to ensure mutual exclusion in a distributed environment. A simple protocol works as follows:

- Let 'a' is a site in quorum 'A'. If 'a' wants to invoke mutual exclusion, it requests permission from all sites in its quorum 'A'.
- Every site does the same to invoke mutual exclusion. Due to the Intersection Property, quorum 'A' contains at least one site that is common to the quorum of every other site.
- These common sites send permission to only one site at any time. Thus, mutual exclusion is guaranteed.

Note that the Minimality property ensures efficiency rather than correctness.

Maekawa's Algorithm

Maekawa divides the processes into a number of subsets of identical size K . Each process i is associated with a unique subset S_i . The subsets satisfy the following three properties:

1. $\forall i, j : 0 \leq i, j \leq n - 1 :: S_i \cap S_j \neq \emptyset$.

Whenever a pair of processes i and j wants to enter their respective CS, a process in $(S_i \cap S_j)$ takes up the role of an arbitrator, and chooses only one of them by sending acknowledgment, and defers the other one.

2. $i \in S_i$. It is only natural that a process gets an acknowledgment from itself for entering the CS. Note that this does not cost a message.
3. Every process i is present in the same number (D) of subsets. The fact that every node acts as an arbitrator for the same number of processes adds to the symmetry of the system.

Maekawa's Algo Rules

- MA1.** To enter its CS, a process i first sends a timestamped request message to every process in S_i .
- MA2.** A process (outside its CS) receiving requests sends an acknowledgment to that process whose request has the lowest timestamp. It locks itself to that process, and keeps all other requests waiting in a request queue. If the receiving process is in its CS, then the acknowledgment is deferred.
- MA3.** A process i enters its CS when receives acknowledgments from each member in S_i .
- MA4.** During exit from its CS, a process sends release messages to all the members of S_i .
- MA5.** Upon receiving a release message from process i , a process unlocks itself, deletes the current request, and sends an acknowledgment to the process whose request has the lowest timestamp.

Example : $k=3; D=3$

$$S_0 = \{0, 1, 2\}$$

$$S_1 = \{1, 3, 5\}$$

$$S_2 = \{2, 4, 5\}$$

$$S_3 = \{0, 3, 4\}$$

$$S_4 = \{1, 4, 6\}$$

$$S_5 = \{0, 5, 6\}$$

$$S_6 = \{2, 3, 6\}$$

The Algorithm

A site S_i executes the following steps to execute the CS.

Requesting the critical section

- (a) A site S_i requests access to the CS by sending REQUEST(i) messages to all sites in its request set R_i .
- (b) When a site S_j receives the REQUEST(i) message, it sends a REPLY(j) message to S_i provided it hasn't sent a REPLY message to a site since its receipt of the last RELEASE message. Otherwise, it queues up the REQUEST(i) for later consideration.

Executing the critical section

- (c) Site S_i executes the CS only after it has received a REPLY message from every site in R_i .

The Algorithm

Releasing the critical section

- (d) After the execution of the CS is over, site S_i sends a RELEASE(i) message to every site in R_i .
- (e) When a site S_j receives a RELEASE(i) message from site S_i , it sends a REPLY message to the next site waiting in the queue and deletes that entry from the queue. If the queue is empty, then the site updates its state to reflect that it has not sent out any REPLY message since the receipt of the last RELEASE message.

Proof

Proof of ME1 (by contradiction). Assume that the statement is false, and two processes i and j ($i \neq j$) enter their critical sections at the same time. For this to happen, every member of S_i must have received the request from i , and every member of S_j must have received the request from j . Since $S_i \cap S_j \neq \emptyset$, there is a process $k \in S_i \cap S_j$ that received requests from both i and j . Per MA2, process k will send acknowledgment to only one of them, and refrain from sending the acknowledgment to the other, until the first process has sent a release signal. Since a process needs an acknowledgment from every member of its subset, both i and j cannot enter their critical sections at the same time. ■

Performance

- Since the size of a request set is $\sqrt{N}$, an execution of the CS requires $\sqrt{N}$ REQUEST, $\sqrt{N}$ REPLY, and $\sqrt{N}$ RELEASE messages, resulting in $3\sqrt{N}$ messages per CS execution.
- Synchronization delay in this algorithm is $2T$. This is because after a site S_i exits the CS, it first releases all the sites in R_i and then one of those sites sends a REPLY message to the next site that executes the CS.

Deadlock in Maekawa's Algo

The proposed algorithm *maekawa1*, however, does not satisfy the safety property ME2, since there is a potential for deadlock. The source of the problem is that no process is clairvoyant — so when a process receives a request, it does not know whether another request with a lower timestamp is on its way. Here is an example. Assume that processes **0**, **1**, **2** have sent request to the members of S_0, S_1, S_2 , respectively. The following scenario is possible:

- From $S_0 = \{0, 1, 2\}$, processes **0**, **2** send *ack* to **0**, but process **1** sends *ack* to **1**
- From $S_1 = \{1, 3, 5\}$, processes **1**, **3** send *ack* to **1**, but process **5** sends *ack* to **2**
- From $S_2 = \{2, 4, 5\}$, processes **4**, **5** send *ack* to **2**, but process **2** sends *ack* to **0**

Thus **0** waits for ¹ **1**, **1** waits for **2**, and **2** waits for **0**. This circular waiting causes deadlock.

There are two possible ways of avoiding a deadlock. The first is to assume a system-wide order of message propagation, as explained below:

[Global FIFO] Every process receives incoming messages in strictly increasing order of timestamps.

Modified Maekawa2

MA1'. To enter its CS, a process i first sends a timestamped request message to every process in S_i . {same as MA1}

MA2'. A process receiving a request can take one of the following three steps when it is outside its CS:

- If lock has not yet been set, then the process sends an acknowledgment to the requesting process with the lowest timestamp, sets its own lock to the id of that process, and send failed messages to the senders of the remaining requests.

Modified Maekawa2 contd..

- If lock is already set, and the timestamp of the incoming request is higher than the timestamp of the locked request, then the incoming request is enqueued and a failed message is sent to the sender.

If lock is already set, but the timestamp of the incoming request is lower than that of the locked request, then the incoming request is queued, and an inquire message is sent to the sender of the locking request. This message is meant to check the current status of the process whose request set the lock.

Handling Deadlocks

- Maekawa's algorithm handles deadlocks by requiring a site to yield a lock if the timestamp of its request is larger than the timestamp of some other request waiting for the same lock.
- A site suspects a deadlock (and initiates message exchanges to resolve it) whenever a higher priority request arrives and waits at a site because the site has sent a REPLY message to a lower priority request.

Deadlock handling requires three types of messages:

- FAILED:** A FAILED message from site S_i to site S_j indicates that S_i can not grant S_j 's request because it has currently granted permission to a site with a higher priority request.
- INQUIRE:** An INQUIRE message from S_i to S_j indicates that S_i would like to find out from S_j if it has succeeded in locking all the sites in its request set.
- YIELD:** A YIELD message from site S_i to S_j indicates that S_i is returning the permission to S_j (to yield to a higher priority request at S_j).

Handling Deadlocks

Maekawa's algorithm handles deadlocks as follows:

- When a $\text{REQUEST}(ts, i)$ from site S_i blocks at site S_j because S_j has currently granted permission to site S_k , then S_j sends a $\text{FAILED}(j)$ message to S_i if S_i 's request has lower priority. Otherwise, S_j sends an $\text{INQUIRE}(j)$ message to site S_k .
- In response to an $\text{INQUIRE}(j)$ message from site S_j , site S_k sends a $\text{YIELD}(k)$ message to S_j provided S_k has received a FAILED message from a site in its request set or if it sent a YIELD to any of these sites, but has not received a new GRANT from it.
- In response to a $\text{YIELD}(k)$ message from site S_k , site S_j assumes as if it has been released by S_k , places the request of S_k at appropriate location in the request queue, and sends a $\text{GRANT}(j)$ to the top request's site in the queue. Maekawa's algorithm requires extra messages to handle deadlocks
- Maximum number of messages required per CS execution in this case is $5\sqrt{N}$.

Token-Based Algorithms

- In token-based algorithms, a unique token is shared among the sites.
- A site is allowed to enter its CS if it possesses the token.
- Token-based algorithms use sequence numbers instead of timestamps. (Used to distinguish between old and current requests.)

Suzuki-Kasami's Broadcast Algorithm

- If a site wants to enter the CS and it does not have the token, it broadcasts a REQUEST message for the token to all other sites.
- A site which possesses the token sends it to the requesting site upon the receipt of its REQUEST message.
- If a site receives a REQUEST message when it is executing the CS, it sends the token only after it has completed the execution of the CS.

continuation..

This algorithm must efficiently address the following two design issues:

(1) How to distinguish an outdated REQUEST message from a current REQUEST message:

- Due to variable message delays, a site may receive a token request message after the corresponding request has been satisfied.
- If a site can not determine if the request corresponding to a token request has been satisfied, it may dispatch the token to a site that does not need it.
- This will not violate the correctness, however, this may seriously degrade the performance.

(2) How to determine which site has an outstanding request for the CS:

- After a site has finished the execution of the CS, it must determine what sites have an outstanding request for the CS so that the token can be dispatched to one of them.

continuation..

The first issue is addressed in the following manner:

- A REQUEST message of site S_j has the form REQUEST(j, n) where n ($n=1, 2, \dots$) is a sequence number which indicates that site S_j is requesting its n^{th} CS execution.
- A site S_i keeps an array of integers $RN_i[1..N]$ where $RN_i[j]$ denotes the largest sequence number received in a REQUEST message so far from site S_j .
- When site S_i receives a REQUEST(j, n) message, it sets $RN_i[j] := \max(RN_i[j], n)$.
- When a site S_i receives a REQUEST(j, n) message, the request is outdated if $RN_i[j] > n$.

continuation..

The second issue is addressed in the following manner:

- The token consists of a queue of requesting sites, Q , and an array of integers $LN[1..N]$, where $LN[j]$ is the sequence number of the request which site S_j executed most recently.
- After executing its CS, a site S_i updates $LN[i] := RN_i[i]$ to indicate that its request corresponding to sequence number $RN_i[i]$ has been executed.
- At site S_i if $RN_i[j] = LN[j] + 1$, then site S_j is currently requesting token.

The Algorithm

Requesting the critical section

- (a) If requesting site S_i does not have the token, then it increments its sequence number, $RN_i[i]$, and sends a REQUEST(i, sn) message to all other sites. (' sn ' is the updated value of $RN_i[i]$.)
- (b) When a site S_j receives this message, it sets $RN_j[i]$ to $\max(RN_j[i], sn)$. If S_j has the idle token, then it sends the token to S_i if $RN_j[i] = LN[i] + 1$.

Executing the critical section

- (c) Site S_i executes the CS after it has received the token.

The Algorithm

Releasing the critical section Having finished the execution of the CS, site S_i takes the following actions:

- (d) It sets $LN[i]$ element of the token array equal to $RN_i[i]$.
- (e) For every site S_j whose id is not in the token queue, it appends its id to the token queue if $RN_i[j] = LN[j] + 1$.
- (f) If the token queue is nonempty after the above update, S_i deletes the top site id from the token queue and sends the token to the site indicated by the id.

Correctness

Mutual exclusion is guaranteed because there is only one token in the system and a site holds the token during the CS execution.

Theorem: A requesting site enters the CS in finite time.

Proof:

- Token request messages of a site S_i reach other sites in finite time.
- Since one of these sites will have token in finite time, site S_i 's request will be placed in the token queue in finite time.
- Since there can be at most $N - 1$ requests in front of this request in the token queue, site S_i will get the token and execute the CS in finite time.

Performance

- No message is needed and the synchronization delay is zero if a site holds the idle token at the time of its request.
- If a site does not hold the token when it makes a request, the algorithm requires N messages to obtain the token. Synchronization delay in this algorithm is 0 or T .